UQAC

Université du Québec
à Chicoutimi

STAGE de BUT - Projet Chemin de garde

Documentation théorique

Algorithme de placement de gardes dans une carte

PAR

Robin ABELA

Noé ARCIER

Rafael CAFÉ

Sacha CHAUVEL

Enzo VILLARET

SOMMAIRE

1. Introduction..... 2
a) Algorithme..... 2
b) Carte (p31: Orthogonal Polygons)..... 2
c) Triangulation (p14-27)..... 2
2. Trous..... 3
a) Détection des trous (orig.) :..... 3
b) Découpe (p126-127: THEOREM 5.1)..... 4
3. Placement des gardes..... 5
a) Quadrilatérisation (origin.)..... 5
b) Cas simples (origin.)..... 5
c) Sous-polygones (p90-114 : Mobile Guards - Orthogonal Polygons)..... 5

1. Introduction

a) Algorithme

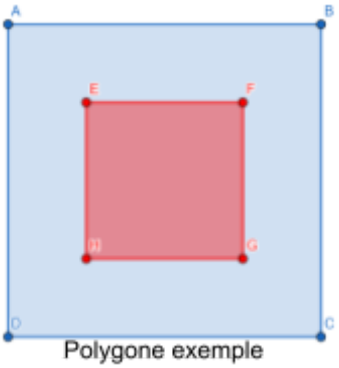
Ce document utilise les théorèmes du livre “Art Gallery Theorems and Algorithms” de Joseph O'ROURKE pour proposer un algorithme permettant de placer des gardes mobiles dans une carte afin qu’ils couvrent l'entièreté de la carte.
Cet algorithme a été conçu pour pouvoir être programmé, certaines parties ne sont pas mathématiquement optimales afin de concilier l’efficacité de l’algorithme avec la simplicité d’implémentation et les possibles limitations techniques.
Pour chaque partie de l’algorithme, il sera indiqué si c’est une adaptation d’un partie du livre(pD-F: [précisions]) ou si c’est une partie inventée(orig.) pour s’adapter aux contraintes de la programmation.

Cet algorithme sera présenté comme une “recette” pour faciliter l’implémentation, les étapes décrites dans l’ordre et leur but expliqué progressivement. Cette “recette” sera accompagnée d’un exemple simple.

Le texte en jaune synthétise les objectifs et le fonctionnement de chaque partie.

b) Carte (p31: Orthogonal Polygons)

La carte choisie doit nécessairement être un polygone orthogonal (ces côtés sont tous rectilignes et parallèles soit à l’axe x, soit à l’axe y), car l’algorithme s’appuie en partie sur des théorèmes du chapitre 2 du livre (Orthogonal Polygons). La carte peut contenir des “trous”, se sont des objets opaques à qui se trouvent à l’intérieur de la carte et qui limitent à la fois les mouvements et la vision des gardes.



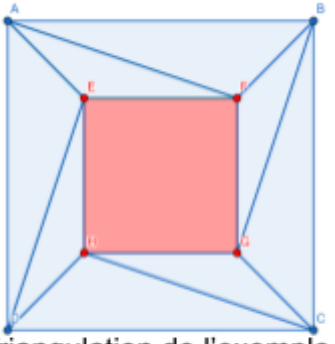
Polygone exemple

À droite, l’exemple qui sera utilisé tout au long du document, c’est une carte carré (en bleu) avec un trou carré en son centre (en rouge).

c) Triangulation (p14-27)

Comme indiqué dans le livre, il existe de multiples algorithmes pour la triangulation, certains sont déjà implémentés dans les moteurs de jeu. Pour cet algorithme, il est nécessaire que la triangulation n’ajoute pas de sommets autres que ceux des polygones représentant la carte et ses trous.

La triangulation ayant déjà été implémentée dans le projet, elle ne sera pas décrite étape par étape ici.



Triangulation de l'exemple

Les informations qui doivent être obtenues sont : triangles, leurs sommets et leurs côtés.
Il doit être possible d’accéder aux sommets d’un côté et aux côtés partant d’un sommet (graphe).Il doit aussi être possible d’accéder aux côtés d’un triangle et à ces voisins ainsi que de savoir quel est leur côté en commun.

2. Trous

Le but de cette partie est de préparer un polygone ayant des trous pour qu'il soit adapté à la suite de l'algorithme, si ce type de polygone n'est pas pris en compte, passer à l'étape 3.

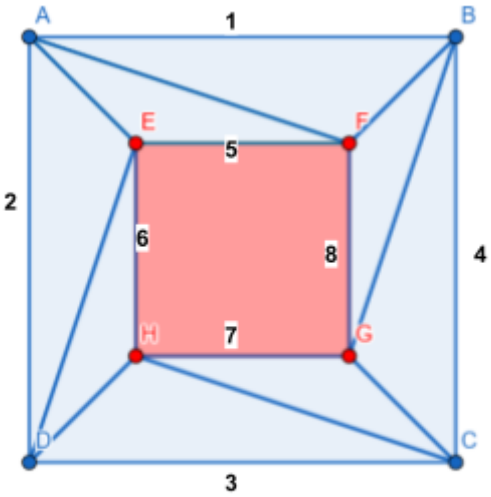
a) <u>Détection des trous (orig.) :</u>

- Pour tous les triangles: identifier les côtés qui ne sont communs à aucun voisins (ils sont donc en contact avec un trou ou l'extérieur), ils forment la liste des côtés de trous.

- Dans cette liste de côtés, en partant de n'importe lequel, créer une liste de côtés spécifique pour le trou n = 1.

- Chercher un côté voisin au premier choisi (sommet en commun) faisant aussi partie de la liste des côtés de trous, ajouter le voisin à la liste des côtés du trou n, chercher dans les voisins du nouveau côté en ignorant le côté précédent, répéter l'opération jusqu'à retomber sur le premier côté.

- Cette liste correspond aux côtés d'un trou, pour les autres trous : enlever les côtés de ce trou à la liste générale des côtés de trous. Incrémenter n de 1 et choisir un nouveau côté dans la liste générale, puis chercher à nouveau les côtés de ce nouveau trou. Répéter le tout jusqu'à ce que la liste générale des côtés des trous soit vide.

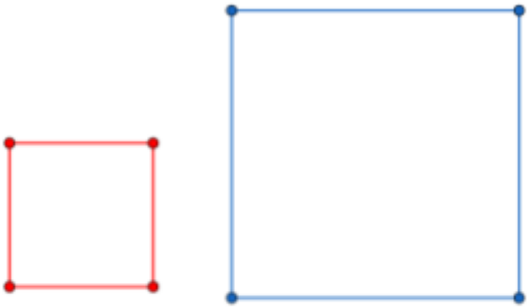


Identification des trous et de leurs côtés :
- Premier trou : 1->2->3->4->1
- Second trou : 5->6->7->8->5

- Parmi ces trous, l'un d'entre eux est en vérité le contour extérieur du polygone. Pour le trouver : pour chaque trou, trouver ses extrêmes sur l'axe x et sur l'axe y : dans les coordonnées de tous les sommets du trou, identifier les minimums et maximums des abscisses et ordonnés

- Ensuite, multiplier la différence entre les extrêmes de l'axe x avec celle des extrêmes de l'axe y. La valeur représente l'aire du rectangle le plus petit possible pouvant contenir le trou.

- Le trou ayant la valeur la plus haute est en vérité le contour extérieur du polygone, puisque tous les autres trous sont contenus dans le polygone et ne peuvent pas être plus grands.



L'aire du plus petit rectangle pouvant contenir le premier trou est plus grande que celle du rectangle correspondant au second trou.
Le premier trou correspond à l'extérieur

À présent, l'extérieur, les trous, mais aussi leurs côtés (et donc leurs sommets) ont été identifiés.

b) Découpe (p126-127: THEOREM 5.1)

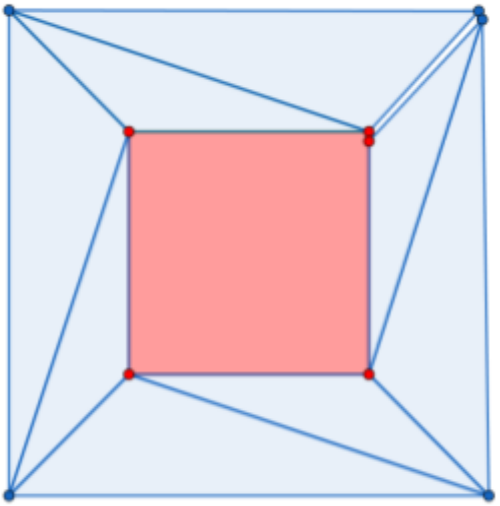
Une fois que les trous et l'extérieur sont identifiés, il s'agit de relier les trous à l'extérieur en coupant le polygone le long des bonnes diagonales. Ce qui permet d'avoir un polygone sans trous pour les étapes suivantes.

- Pour chaque trou (qui ne soit pas le contour extérieur) : parcourir les sommets du trou, pour chaque sommet : vérifier si un côté le relie à point du contour extérieur, dès qu'un tel côté est trouvé arrêter la recherche pour ce trou. Si aucun côté adéquat n'est trouvé, attendre la prochaine itération.

- Pour que le trou puisse être considéré comme faisant partie de l'extérieur : les triangles voisins dont le côté commun est celui choisi ne sont plus voisins, pour chacun ce côté les relie désormais à l'extérieur (comme si passage infiniment petit s'était ouvert pour relier le trou à l'extérieur).

- Répéter pour tous les trous jusqu'à ce qu'ils ne restent uniquement que des trous ne pouvant être reliés directement avec l'extérieur.

- S'il reste des trous, ceux-ci ne peuvent pas être reliés directement car d'autres trous se trouvent entre eux et l'extérieur. Pour régler le problème : ajouter tous les sommets des trous précédemment reliés dans la liste des sommets de l'extérieur puis parcourir à nouveau les trous.



La diagonale en haut à droite relie le trou avec l'extérieur

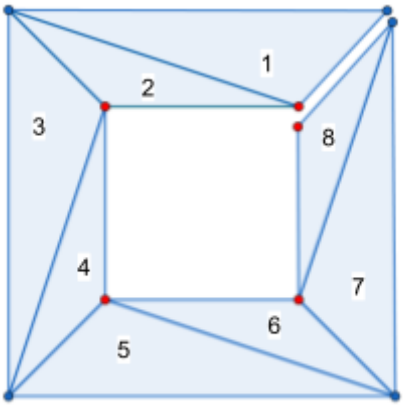
- Itérer jusqu'à ce que tous les trous soient reliés avec l'extérieur.

Avec cette méthode, un polygone unique et sans trou est obtenu.

3. Placement des gardes

a) Quadrilatérisation (origin.)

Cette étape rassemble les triangles de la triangulation 2 par 2 pour qu'ils forment des quadrilatères.



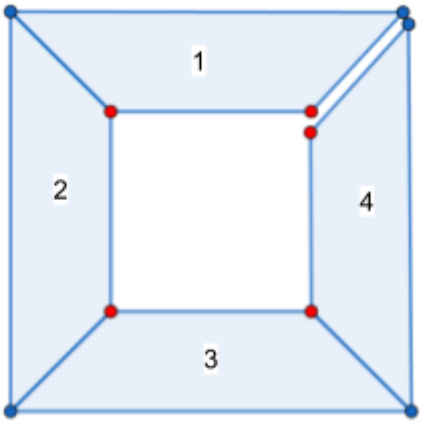
Triangles du polygone

Si le livre décrit de nombreuses manières de quadrilatèriser un polygone, pour simplifier l'implémentation, nous avons choisi une méthode simplifiée:

- Considérant le graphe représentant tous les triangles du polygone et donc chaque arête correspond à un lien de voisinage entre triangles (côté un commun). On appelle une feuille un triangle n'ayant qu'un seul voisin.

- En partant d'une feuille, si son unique voisin n'a qu'un seul autre voisin, alors créer un quadrilatère rassemblant les 2 triangles et enlever les triangles de la liste des triangles ou de leurs voisins. Si le voisin a plus d'un autre voisin (total de 3 donc), ignorer pour l'instant.

- Sélectionner une nouvelle feuille et recommencer. La suppression des triangles fusionnés en quadrilatères devrait permettre l'apparition de nouvelles feuilles.



Quadrilatères du polygone

Les quadrilatères ainsi obtenus forment eux aussi un graphe dont chaque sommet est un quadrilatère est chaque arête un lien de voisinage (côté en commun) avec un autre quadrilatère.

b) Cas simples (origin.)

- Si le polygone n'est composé que d'un quadrilatère, il suffit de placer un garde le long d'une des ses diagonales.

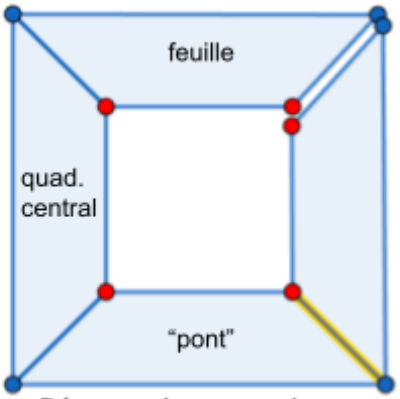
- Si le polygone est composé de deux quadrilatères, il suffit de placer un garder sur le côté commun aux deux quadrilatères.

c) Sous-polygones (p90-114 : Mobile Guards - Orthogonal Polygons)

Pour placer des gardes dans un polygone complexe, nous allons isoler des sous-polygones, ceux-ci doivent pouvoir être couverts par un seul garde mobile, mais aussi être le plus grand possible pour limiter le nombre de gardes.

- Pour identifier un sous-polygone : en partant d'une feuille (quad. n'ayant qu'un seul voisin), son unique voisin sera considéré comme le quadrilatère central du sous polygone.

- Pour vérifier que le sous-polygone soit valide : vérifier que le quadrilatère central n'est pas plus d'un voisin qui ne soit pas une feuille (tous ses voisins sauf un ou aucun ne doivent pas avoir



Découpe du sous-polygone

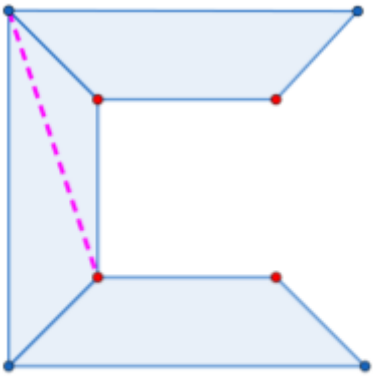
d'autres voisins), si ce n'est pas le cas chercher un autre sous-polygone.

- Si le quadrilatère central n'a que des voisins feuilles, alors placer un garde sur une de ses diagonales puis arrêter la recherche de sous-polygones, car le polygone est à présent entièrement couvert (cas de sortie).

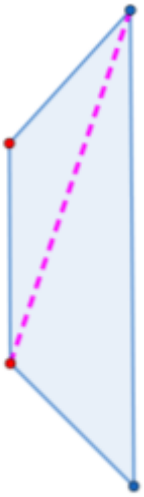
- Si le quadrilatère central a un voisin non feuille et que celui-ci n'a qu'un seul autre voisin, alors c'est le "pont" entre le sous-polygone et le reste du polygone. Pour isoler le sous-polygone, il faut couper le côté du "pont" opposé au quadrilatère central. Ce côté est le seul côté du "pont" dont aucun des sommets n'est commun avec le quadrilatère central.

- Si le voisin non feuille du quadrilatère central a plusieurs autres voisins, alors le côté à couper est le côté commun entre le quadrilatère central et son voisin non feuille.

- Ensuite, placer un garde sur une diagonale du quadrilatère central et retirer les quadrilatères du sous-polygone du polygone général, puis répéter jusqu'à tomber sur un cas simple ou sur le cas de sortie d'un quadrilatère central n'ayant que des voisins feuilles.



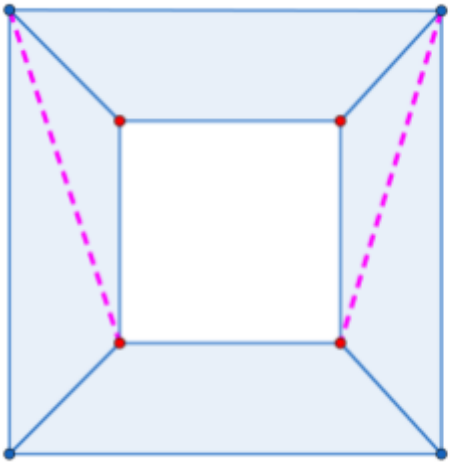
1er sous-polygone: Garde sur la diagonale du quadrilatère central



2eme sous-polygone: Cas simple, garde sur la diagonale du quadrilatère unique

À présent, tous les gardes ont été placés, si vous vous êtes projetés dans le cadre d'un jeu vidéo, vous remarquez que les gardes ne peuvent pas couvrir l'entièreté de la carte s'ils ne regardent pas des deux côtés quand leur parcours fini sur un angle.

Dans le cadre d'un garde humain moyen, il faut donc qu'il dépasse un peu cet angle, mais il faut aussi également que le garde balaie largement ses alentours pendant sa patrouille car les "gardes" de cet algorithme sont considérés comme ayant un champ de vision à 360° et sans limite de distance.



Placement final des gardes (en violet) dans le polygone original.

La carte n'est couverte entièrement qui si au moins un des deux gardes regarde derrière le coin au sud de son chemin de patrouille.